

Multimodal Rag

From Data to Solutions
ETH Zürich · 14. August 2026

Ana Sofia Luna



Agenda

- 1 Motivation
- 2 Image processing with CLIP
- 3 Processing semi structured data
- 4 Multimodal ingestion pipelines

01

Motivation

Intuitive solution

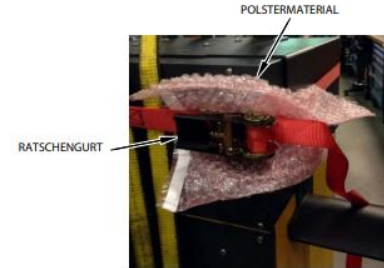


Transcribe



	A	B	C	D
1	Customer_id	Product	Sales channel	Cost
2	1001	Lamps	Retail	800
3	1002	stool	Online	3200
4	1003	Table	online	3000
5	1004	Stool	Retail	4500
6	1005	desk	Online	3700
7	1006	Lamps	online	2500
8	1007	cables	Retail	1700
9	1008	desk	Online	2300

Convert to text



LLM to text



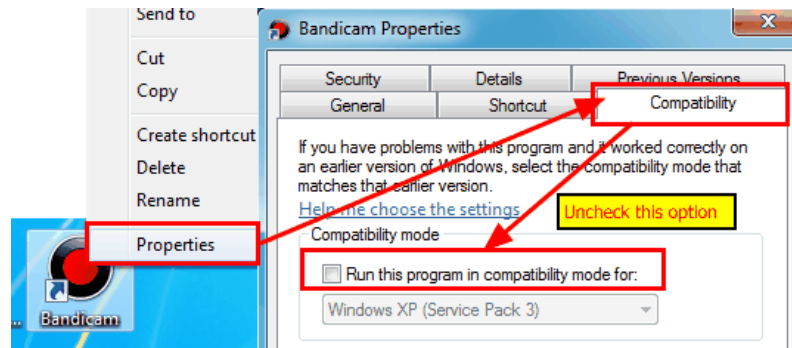
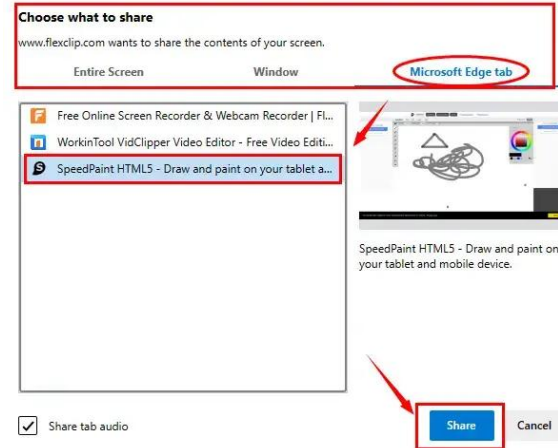
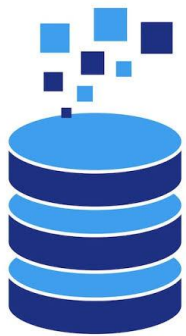
Vectorize



Intuitive solution



Transcribe



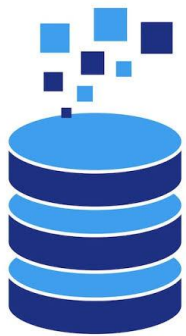
Problem

- Background noise can corrupt video
- Quality of content may be unclear to LLM
- Relies on a reliable transcription

Intuitive solution

	A	B	C	D
1	Customer_id	Product	Sales channel	Cost
2	1001	Lamps	Retail	800
3	1002	stool	Online	3200
4	1003	Table	online	3000
5	1004	Stool	Retail	4500
6	1005	desk	Online	3700
7	1006	Lamps	online	2500
8	1007	cables	Retail	1700
9	1008	desk	Online	2300

Transcribe



	A	B	C	D
1	Customer_id	Product	Sales channel	Cost
2	1001	Lamps	Retail	800
3	1002	stool	Online	3200
4	1003	Table	online	3000
5	1004	Stool	Retail	4500
6	1005	desk	Online	3700
7	1006	Lamps	online	2500
8	1007	cables	Retail	1700
9	1008	desk	Online	2300

Vector1 = [customer_id product sales cost]

Vector1 = [1002 stool Online 3200]

Problem

- Text embeddings treat rows as words, not records
- The structure and ability to support computation and lookups is gone

Intuitive solution

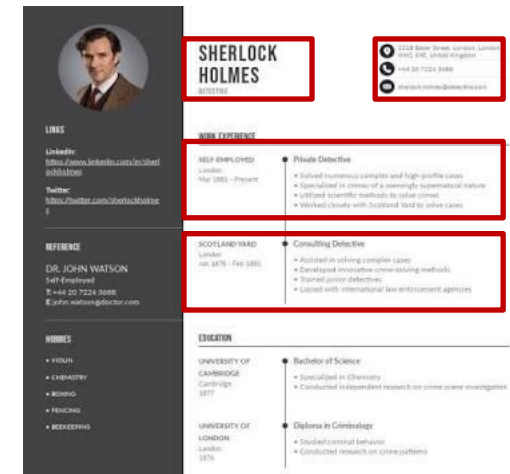


Vectorize



Problem

- Text embeddings do not take into consideration the structure it should be read in
- There is a combination of images that are completely ignored when processing

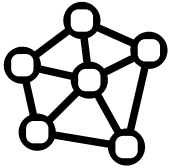


Solution

Data Type



Representation



How it is stored



Images



Image embeddings (using models like CLIP)

[0.12, -0.45, 0.23, ...]

Vector Database
(Image embeddings)

Tables

	A	B	C	D
1	Customer_id	Product	Sales channel	Cost
2		1001 Lamps	Retail	800
3		1002 stool	Online	3200
4		1003 Table	online	3000
5		1004 Stool	Retail	4500
6		1005 desk	Online	3700
7		1006 Lamps	online	2500
8		1007 cables	Retail	1700
9		1008 desk	Online	2300

Structured (using DataFrame/SQL Tables)

id	product	channel	cost
1001	Lamps	Retail	800
1002	Stool	Online	3200
...	...		

Relational DB / Data
warehouses or data lakes

Video

Frames + Audio transcript
+ metadataVector Database for
frames (image
embeddings)

+

Traditional vector Data
base for transcripts

Tables

SHERLOCK HOLMES	
LIST 1. Sherlock Holmes is a fictional character created by the British author Sir Arthur Conan Doyle. 2. He is a detective who lives at 221B Baker Street in London. 3. He is a brilliant investigator and a master of disguise.	HOW TO FIND OUT MORE Read the original stories and biographies. Watch the TV series and movies. Visit the Sherlock Holmes website.

Parsed structure
information (JSON)

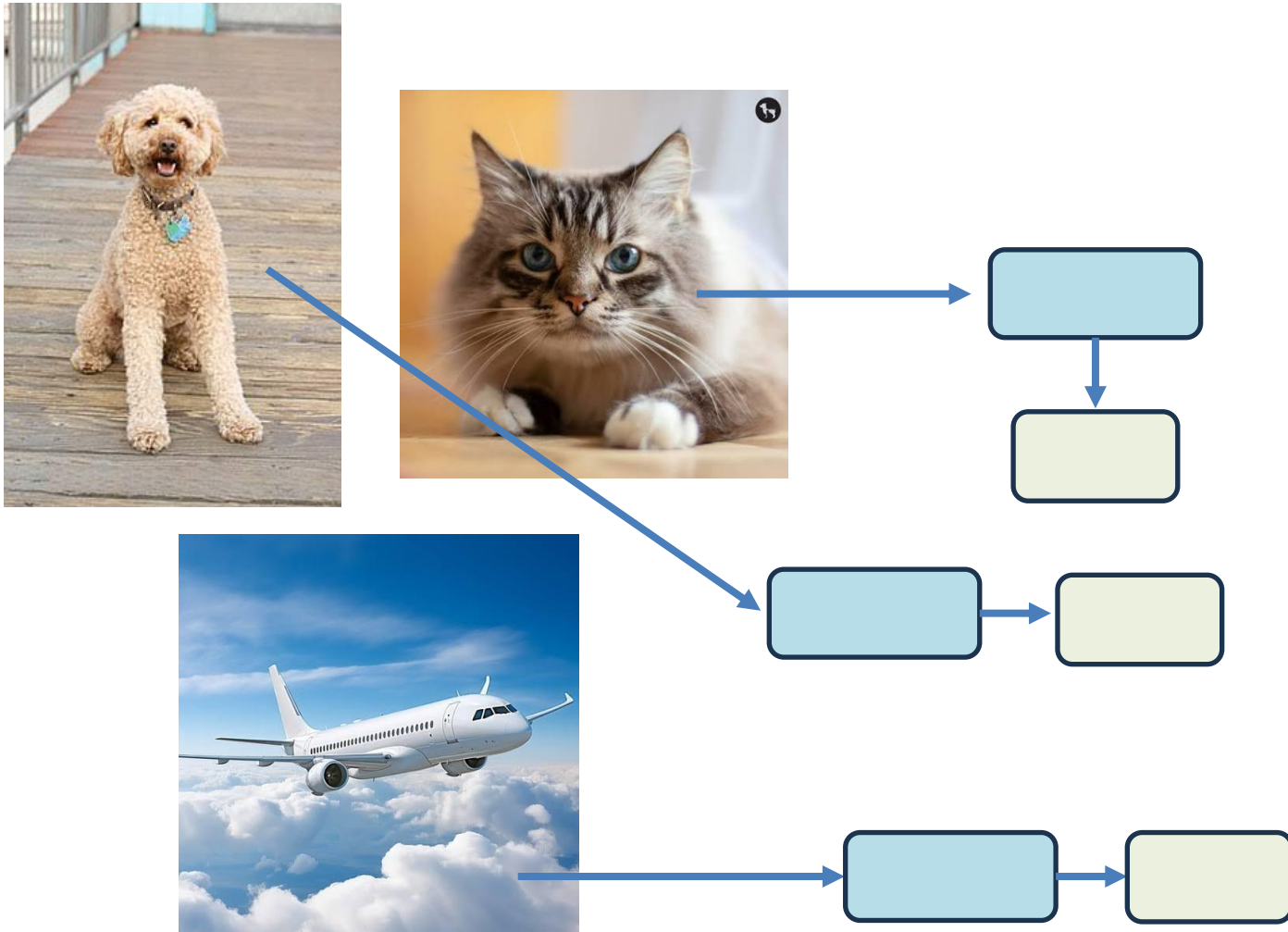
```
{
  "name": "Einstein"
}
```

Document Database /
JSON store

02

Image processing with CLIP

How Do We Search Images?



Naive Solution

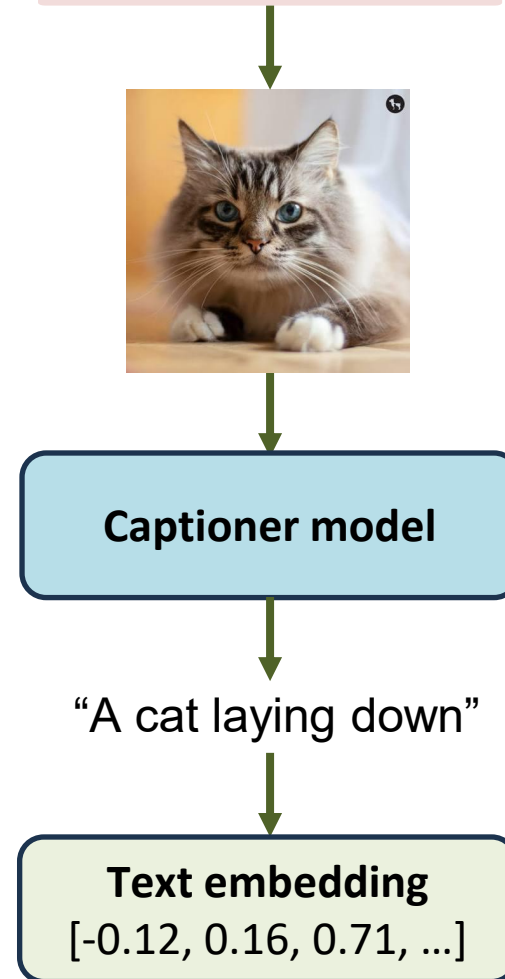


Image captioning

Why this fails

- **Information loss:** Captioners tend to oversimplify.
(Imagine we were searching for a specific breed of cat)
- **Fixed description:** Captions omit nuances like background objects
- **Error propagation:** If the captioning makes a mistake, the search will be incorrect

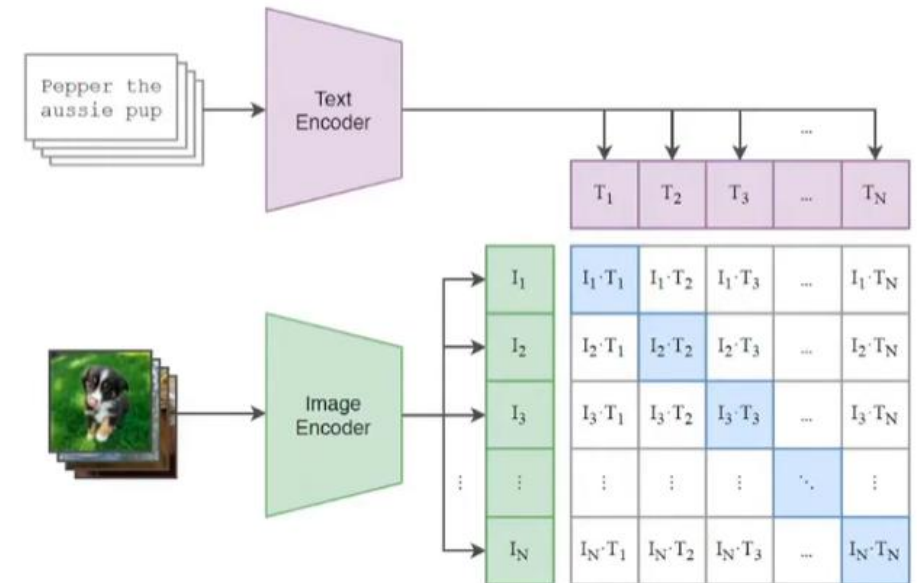
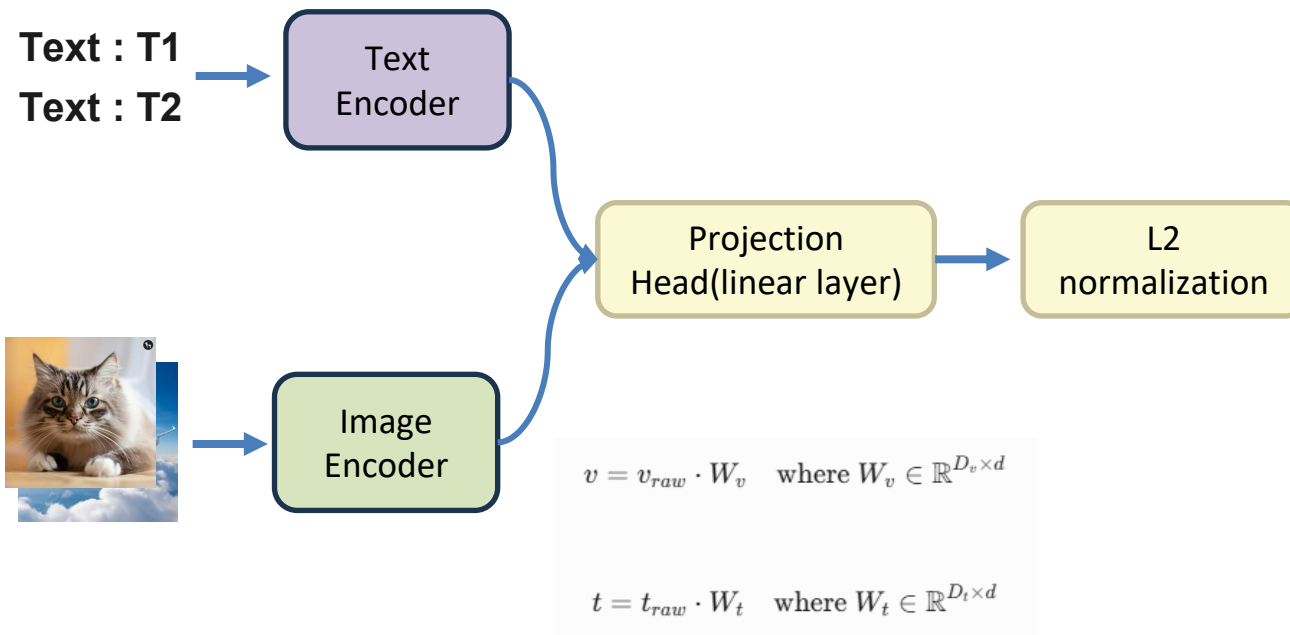
What do we need?

- **Images to be embedded directly**
- **Image embedding to live in a shared vector space that allows us to search for semantic proximity**

CLIP: How can images live in the same space

CLIP (Contrastive Language Image Pre training) has:

Two Independent Encoders



CLIP Architecture



Text : T1



Text : T2

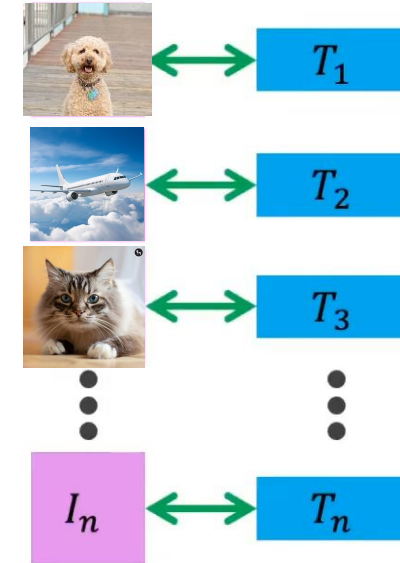


Text : T2

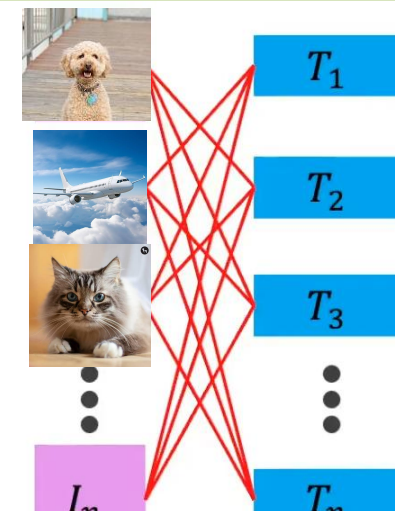
Goal

Find which text (as a whole) goes with which image

Goal: increase the cosine similarity of the correct pairs

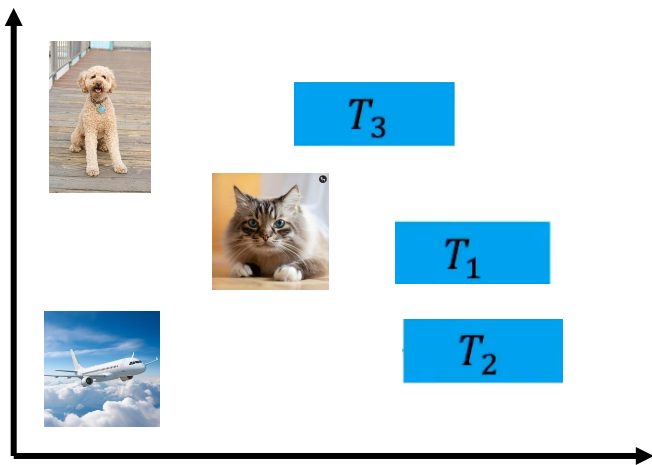


Goal: reduce cosine similarity of $n^2 - n$ incorrect pairs



CLIP Architecture

Before training



Cosine similarity

Cosine similarity for all N*N combinations

$$\text{logit}_{ij} = \frac{l_i \cdot T_j}{\|l_i\| \|T_j\|} \cdot e^\tau$$

Cosine similarity for all N*N combinations

e^τ is the learnable temperature that scales the logits

Softmax

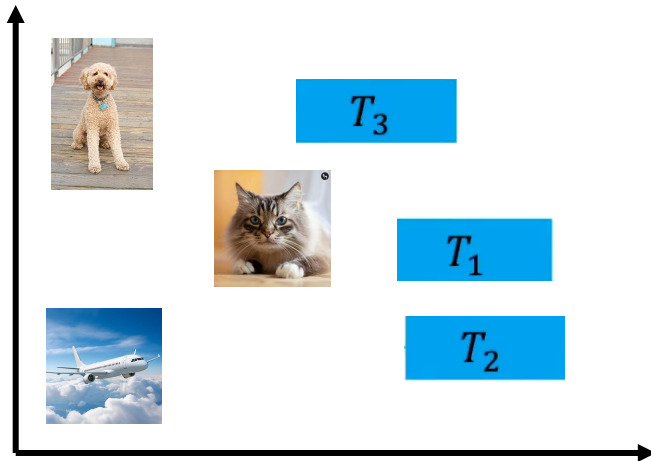
Softmax converts the logit scores into probabilities

$$\text{logit}_{ij} = \frac{e^{\text{logit}_{ij}}}{\sum_k e^{\text{logit}_{ik}}}$$

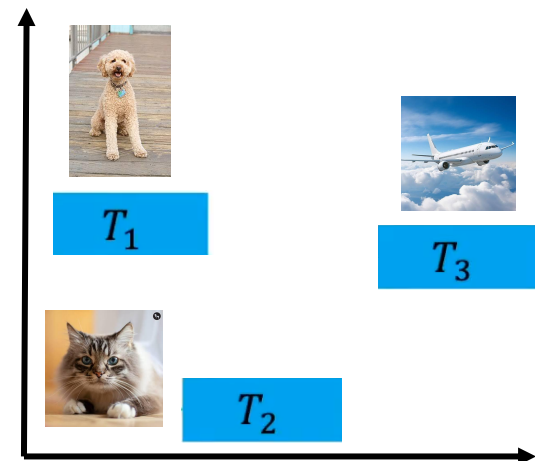
	T_1	T_2	T_3
I_1 Dog	0.95	0.12	0.18
I_2 Plane	0.11	0.93	0.20
I_3 Cat	0.22	0.15	0.91

CLIP Architecture

Before training



After training



Loss function: cross-Entropy

The cross entropy is calculated for every image text and text image pair

$$L_{\text{image} \rightarrow \text{text}} = -\frac{1}{N} \cdot \sum_i \log P(T_i | I_i)$$

$$L_{\text{text} \rightarrow \text{image}} = -\frac{1}{N} \cdot \sum_i \log P(I_i | T_i)$$

$$L_{\text{total}} = -\frac{L_{\text{image} \rightarrow \text{text}} + L_{\text{text} \rightarrow \text{image}}}{2}$$

	T_1	T_2	T_3
I_1 Dog	0.95	0.12	0.18
I_2 Plane	0.11	0.93	0.20
I_3 Cat	0.22	0.15	0.91

CLIP Inference / zero shot classification

No Task-Specific Fine-Tuning Required

How do we classify an unseen image into custom classes ?

Suppose the task: Classifying the severity of damage from a car accident



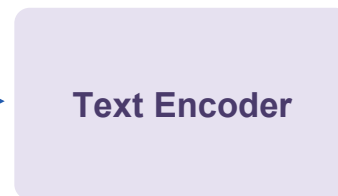
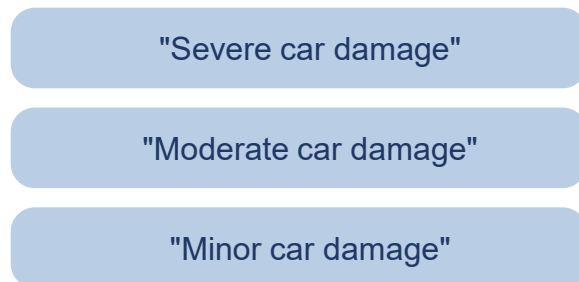
Minor damage



Severe damage

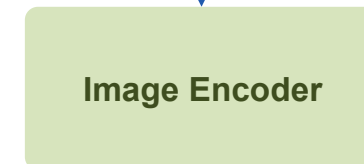
1. Construct text prompt: “a photo of ...”
2. Pass all candidate prompts through encoder
3. Pass input image through encoder

Classification text prompt(chosen at inference time)



T1...T3

Input image



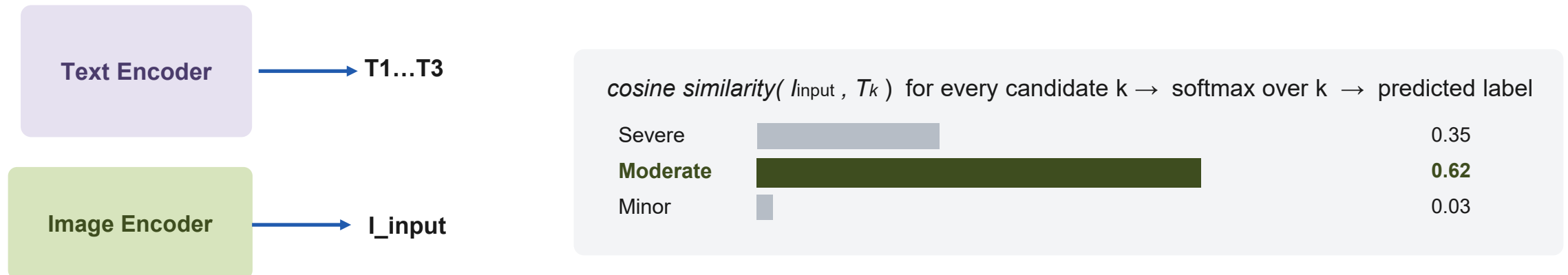
I_input

CLIP Inference / zero shot classification

No Task-Specific Fine-Tuning Required

Suppose the task: Classifying the severity of damage from a car accident

1. Construct text prompt: “a photo of ...”
2. Pass all candidate prompts through encoder
3. Pass input image through encoder
4. Compute cosine similarity and loss



03

Processing Semi-structured data

What is considered “Semi-Structured”?

Structured

Fixed schema, rows & columns.

	A	B	C	D
1	Customer_id	Product	Sales channel	Cost
2	1001	Lamps	Retail	800
3	1002	stool	Online	3200
4	1003	Table	online	3000
5	1004	Stool	Retail	4500
6	1005	desk	Online	3700
7	1006	Lamps	online	2500
8	1007	cables	Retail	1700
9	1008	desk	Online	2300

- SQL tables
- CSV / spreadsheet exports
- DataFrames

Semi-structured

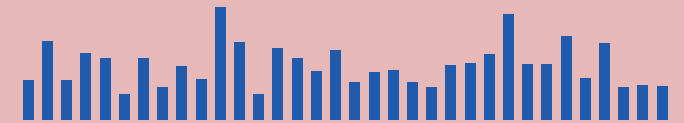
Has organizing structure headings, tags, tables, relationships. The layout itself carries meaning.

The image shows a sample invoice form. It includes a header section with 'Company Name' and 'INVOICE' in large blue text. Below this, there are fields for 'Street Address', 'City, ST ZIP', 'Phone', 'Fax', and 'Website'. To the right, there is a table with 'DATE' (12/9/2019), 'INVOICE #' (123456), 'CUSTOMER ID' (123), and 'DUE DATE' (1/8/2020). Below the header, there is a 'BILL TO' section with fields for 'Name', 'Company Name', 'Street Address', 'City, ST ZIP', and 'Phone'. At the bottom, there is a table with columns 'DESCRIPTION', 'TAXED', and 'AMOUNT'. The table contains rows for 'Service Fee' (230.00), 'Labor: 5 hours at \$75/hr' (375.00), and 'Parts' (345.00). The total amount is 950.00.

- PDF / Word reports (text + tables + figures)
- Invoices, slide decks, forms

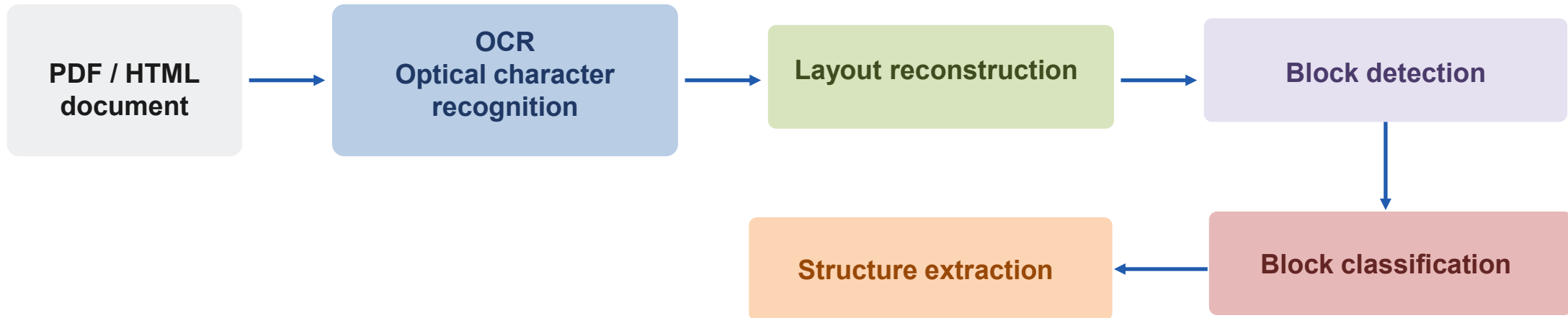
Unstructured

No inherent structure to lean on.



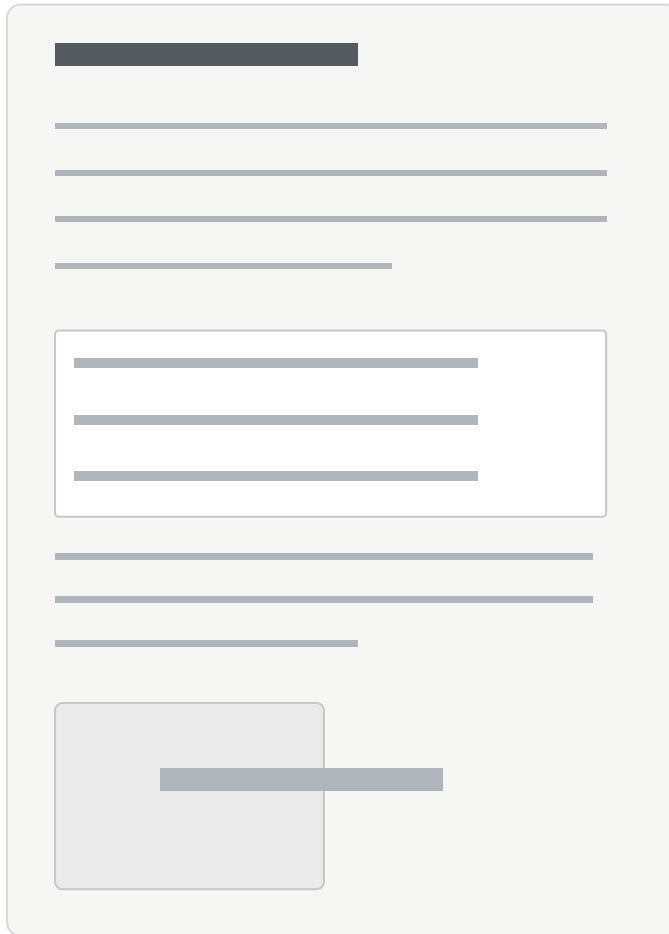
- Raw prose
- A single photo
- An audio clip

Document parsers



Step 1 of 6 · Turning Pixels Into Text (OCR)

scanned page / PDF



1 · Text detection

A detection model scans the page image and extracts textual characters along with their precise bounding box coordinates ($X_{\min}$, $Y_{\min}$, $X_{\max}$, $Y_{\max}$) before it knows what any of it says.

2 · Text recognition

For each detected box, a recognition model reads the pixels inside it and outputs the character sequence e.g. the crop under the "Total" box becomes the string "Total".

Problem

OCR detects and recognises, if the document is low resolution, has an unusual font or is blurry it may not be able to detect or recognise any text.

Step 2 of 6 · Every Token Gets a Position

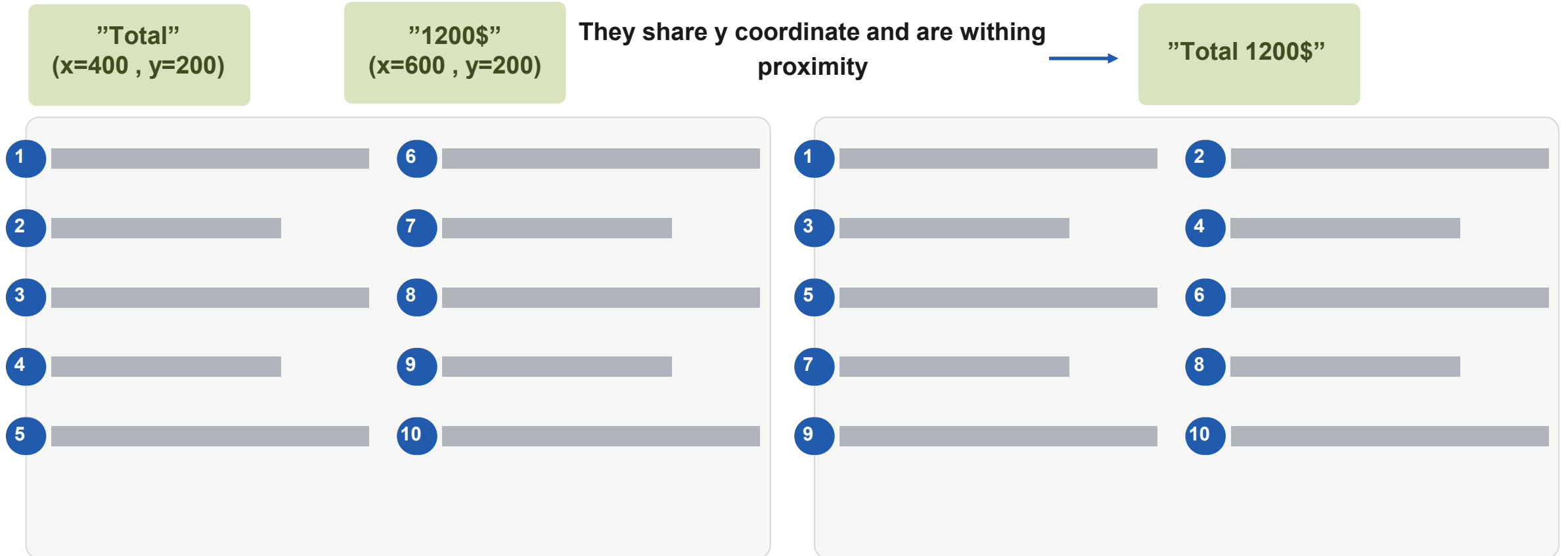
OCR not only extracts text, it also outputs a bounding box and a confidence score that position is what every later step depends on for each token.

Knowing **where** each token is will allow us to rebuild its structure based on proximity.

token	bbox (x, y, w, h)	confidence
"Date:"	(120, 300, 50, 16)	0.99
"03/14/2026"	(175, 300, 90, 16)	0.91
"Total"	(120, 340, 60, 18)	0.98
"\$124.50"	(185, 340, 70, 18)	0.95
"Tota1" (misread)	(120, 380, 58, 18)	0.54

Step 3 of 6 · Reconstructing Reading Order

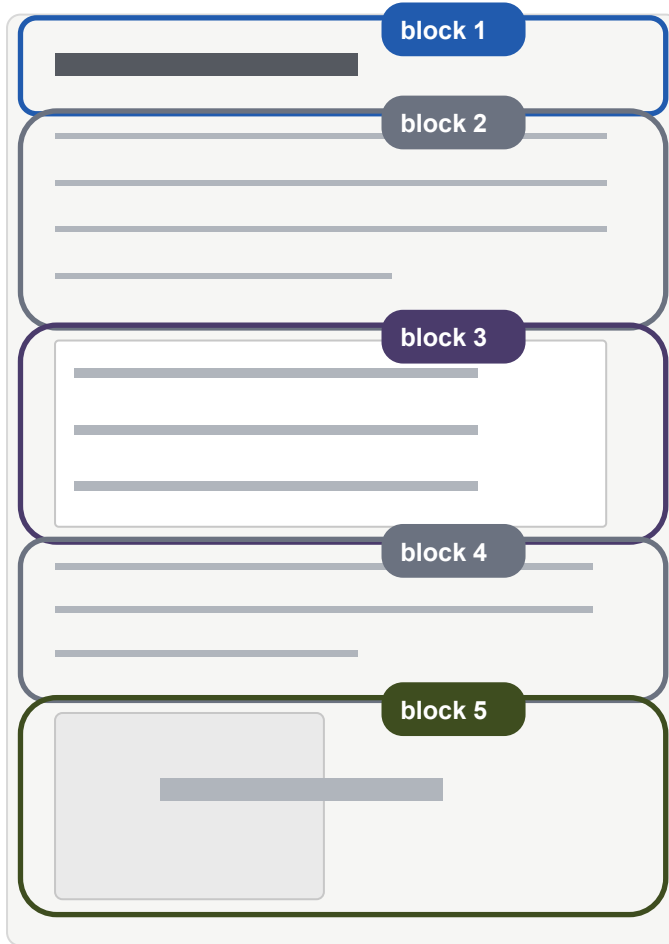
Coordinates let us use spatial and distance thresholds to reconstruct lines, reading order and multi column flows.



Correct: finish column 1 top-to-bottom, then column 2.

Naive top-to-bottom-by-y-coordinate: interleaves the two columns and scrambles meaning.

Step 4 of 6 · Detecting Blocks



A layout-detection model (trained on labeled document images, e.g. PubLayNet / DocLayNet-style data) looks at the whole page and draws a box around each spatially coherent region using whitespace gaps, alignment, and rule lines as cues.

At this stage, blocks are just regions — not yet labeled by type.

Step 5 of 6 · Classifying Each Block



Now each block gets a semantic label Title / Paragraph / Table / Figure / Caption / Header / Footer so downstream steps know how to treat it. Block classification is done with object detection neural networks like RT-DETR or LayoutLM to classify what type of content is in each block.

- **What the classifier looks at:** the block's text content, its font size/weight, and its position on the page.
- **Why it matters:** a table's rows shouldn't be read as prose, and a figure caption shouldn't be merged into the paragraph above it.

Step 6 of 6 · Assembling the Structure

The classified, ordered blocks are walked in reading order and assembled into a nested tree — heading levels define the nesting.

Title	block 1, Claims Report
Paragraph	block 2, text
Table	block 3, rows: “amount”, “state”, “part” →
Paragraph	block 4, text
Figure	block 5, img_003

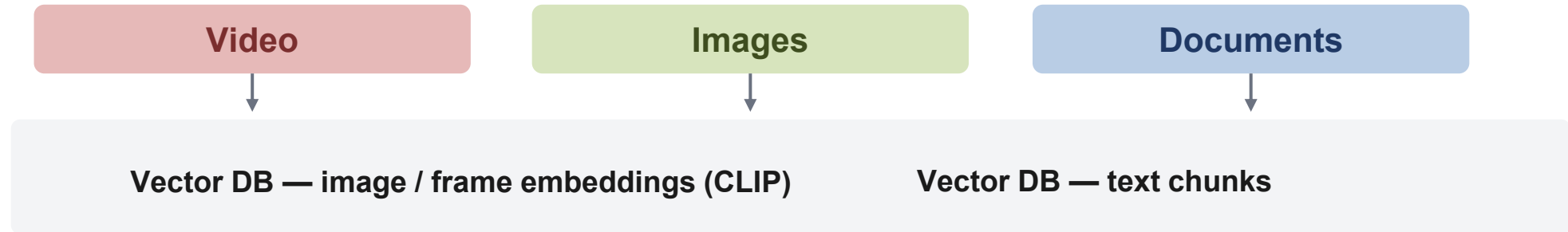
```
{
  "section": "Claims Report",
  "subsection": "Vehicle Damage",
  "blocks": [
    {"type": "paragraph", "text": "..."},
    {"type": "table", "rows": [ {"amount": 6500,
      "state": "CA", "part": "rear_bumper"} ]},
    {"type": "paragraph", "text": "..."},
    {"type": "figure", "caption": "Fig. 2",
      "image_ref": "img_003"}
  ]
}
```

03

Multimodal ingestion pipelines

An End-to-End Multimodal RAG Pipeline

INGESTION



QUERY

